

Taller: CampusTasks — entorno, actualización y eliminación de tareas

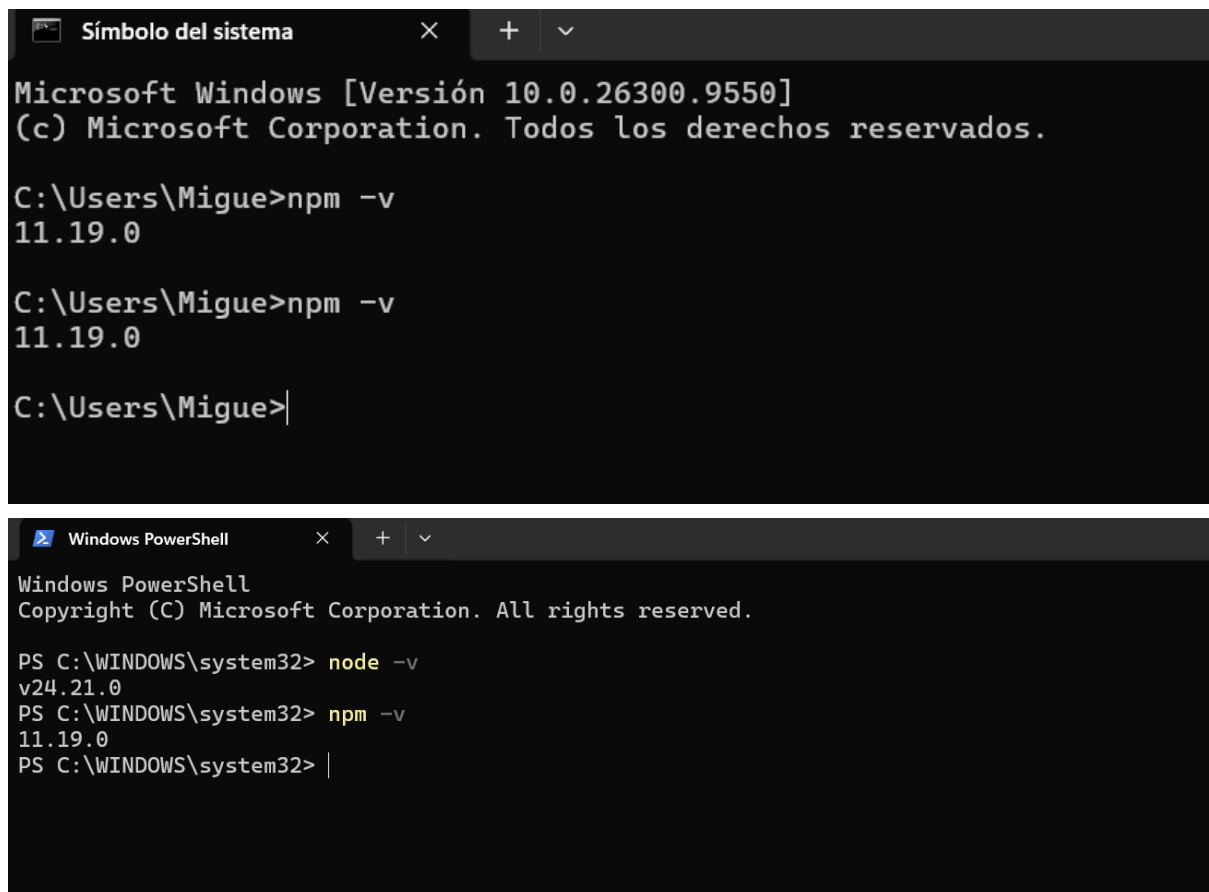
Aura Maria Pelaez Luna - 202459422
Miguel Angel Uribe Perez - 202459430
Valentina Valencia Lopez -202459626
Docente: Kevin Steven Victoria Ospiana

Paso 1. Instalar Node.js 24.x

1. Versiones node.js

- node -v debe mostrar algo que empiece por v24 (por ejemplo v24.x.x).
- npm -v debe mostrar un número de versión.

La terminal con ambas versiones, node -v y npm -v



```
Microsoft Windows [Versión 10.0.26300.9550]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Migue>npm -v
11.19.0

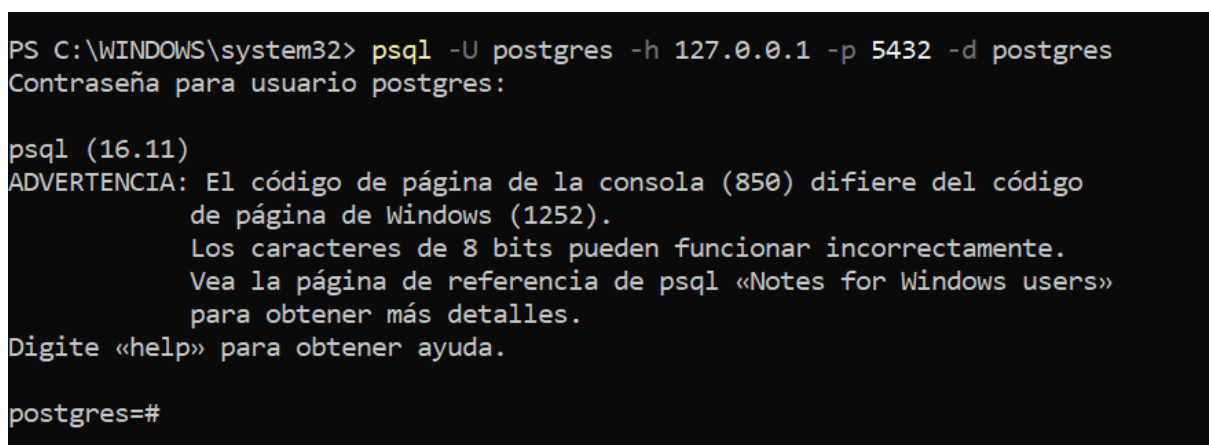
C:\Users\Migue>npm -v
11.19.0

C:\Users\Migue>

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> node -v
v24.21.0
PS C:\WINDOWS\system32> npm -v
11.19.0
PS C:\WINDOWS\system32> |
```

Paso 2. Instalar PostgreSQL



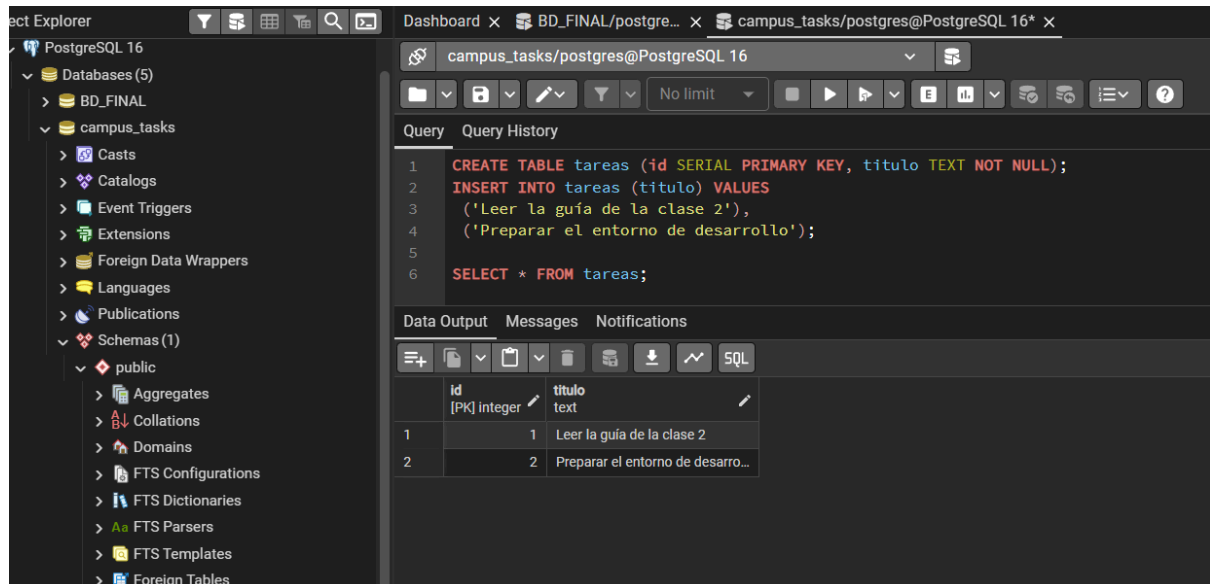
```
PS C:\WINDOWS\system32> psql -U postgres -h 127.0.0.1 -p 5432 -d postgres
Contraseña para usuario postgres:

psql (16.11)
ADVERTENCIA: El código de página de la consola (850) difiere del código
de página de Windows (1252).
Los caracteres de 8 bits pueden funcionar incorrectamente.
Vea la página de referencia de psql «Notes for Windows users»
para obtener más detalles.
Digite «help» para obtener ayuda.

postgres=#
```

Paso 3. Crear la base de datos, la tabla y los datos iniciales

Creación y configuración inicial de la base de datos `campus_tasks` en PostgreSQL, incluyendo la definición de la tabla `tareas` y la inserción de los datos iniciales, verificado exitosamente tanto desde la interfaz gráfica de pgAdmin como mediante comandos en la consola.

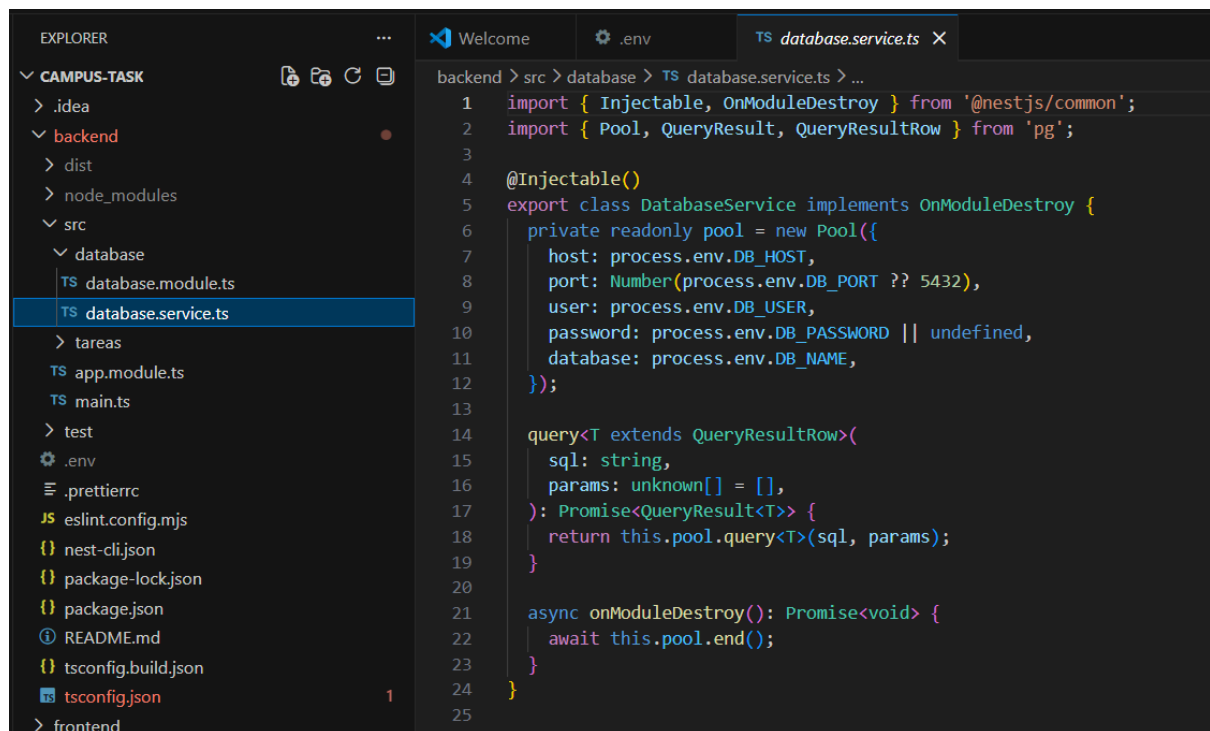


```
postgres=# CREATE DATABASE campus_tasks;
CREATE DATABASE
postgres=# \c campus_tasks
Ahora está conectado a la base de datos «campus_tasks» con el usuario «postgres».
campus_tasks=# CREATE TABLE tareas (id SERIAL PRIMARY KEY, titulo TEXT NOT NULL);
CREATE TABLE
campus_tasks=# INSERT INTO tareas (titulo) VALUES ('Leer la guía de la clase 2'), ('Preparar el entorno de desarrollo');
INSERT 0 2
campus_tasks=# SELECT * FROM tareas;
 id |          titulo
-----+-----
  1 | Leer la guía de la clase 2
  2 | Preparar el entorno de desarrollo
(2 filas)
```

```
campus_tasks=# \q
PS C:\WINDOWS\system32>
```

Paso 4. Revisar de dónde lee el backend la configuración

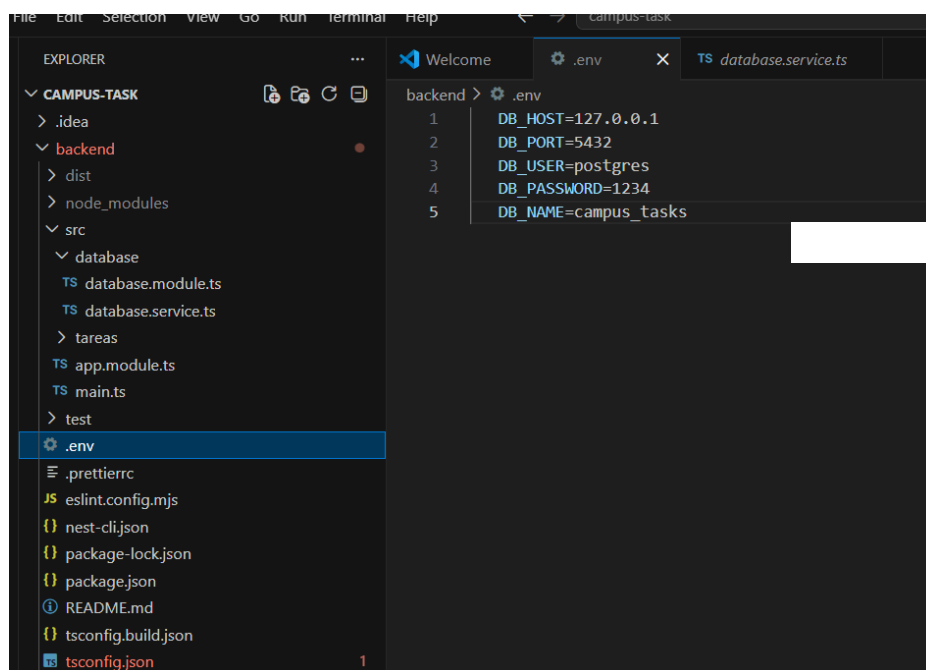
Revisión de database.service.ts: variables de entorno que usa el backend para conectarse a PostgreSQL.



```
1 import { Injectable, OnModuleDestroy } from '@nestjs/common';
2 import { Pool, QueryResult, QueryResultRow } from 'pg';
3
4 @Injectable()
5 export class DatabaseService implements OnModuleDestroy {
6   private readonly pool = new Pool({
7     host: process.env.DB_HOST,
8     port: Number(process.env.DB_PORT ?? 5432),
9     user: process.env.DB_USER,
10    password: process.env.DB_PASSWORD || undefined,
11    database: process.env.DB_NAME,
12  });
13
14  query<T extends QueryResultRow>(
15    sql: string,
16    params: unknown[] = [],
17  ): Promise<QueryResult<T>> {
18    return this.pool.query<T>(sql, params);
19  }
20
21  async onModuleDestroy(): Promise<void> {
22    await this.pool.end();
23  }
24 }
25
```

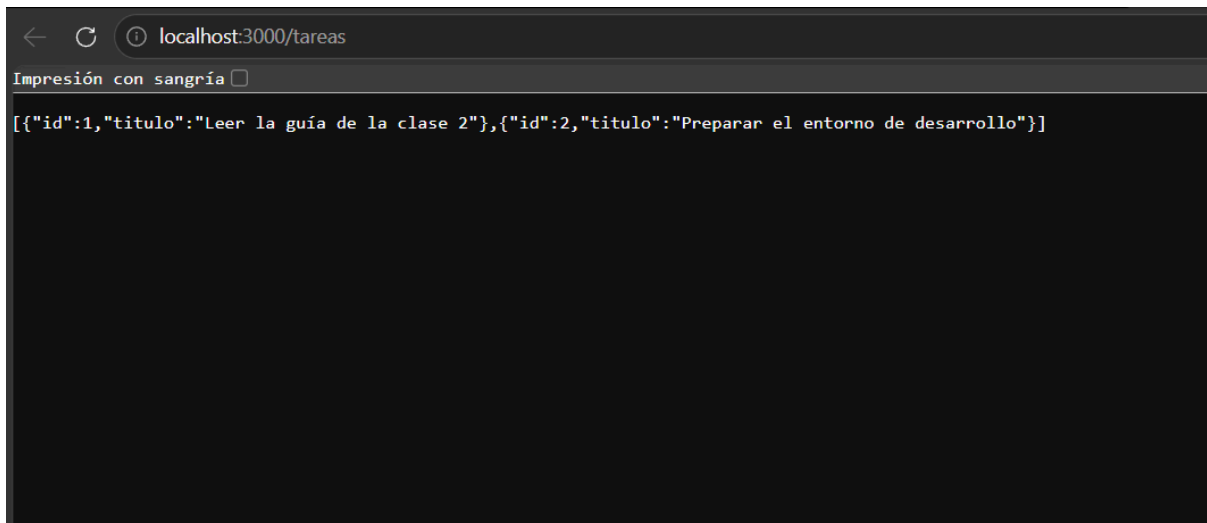
Paso 5. Crear el archivo backend/.env

Archivo backend/.env con las variables de conexión (contraseña oculta).

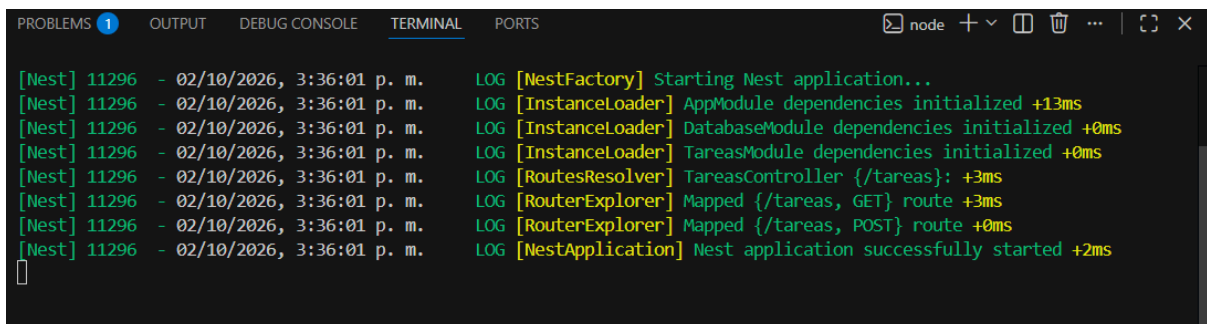


```
1 DB_HOST=127.0.0.1
2 DB_PORT=5432
3 DB_USER=postgres
4 DB_PASSWORD=1234
5 DB_NAME=campus_tasks
```

Paso 6. Instalar y arrancar el backend



Comprobación en el navegador: GET /tareas devuelve las dos tareas iniciales.



Backend NestJS arrancando con `npm run start:dev`.

Paso 7. Instalar y arrancar el frontend

1. Terminal del frontend

Instalación exitosa de dependencias y despliegue del servidor de desarrollo del frontend (npm ci y npm start / ng serve), logrando la compilación correcta de la aplicación Angular y dejándola en ejecución en <http://localhost:4200/>.

```
PS C:\Trabajos\Desarrollo 2\campus-task> cd frontend
>> npm ci
>> npm start
npm warn install-scripts lmd@3.4.2 (install: node-gyp-build-optional-packages)
npm warn install-scripts msgpackr-extract@3.0.4 (install: node-gyp-build-optional-packages)
npm warn install-scripts Run `npm install-scripts ls` to review, or `npm install-scripts approve <pkg>` to allow.

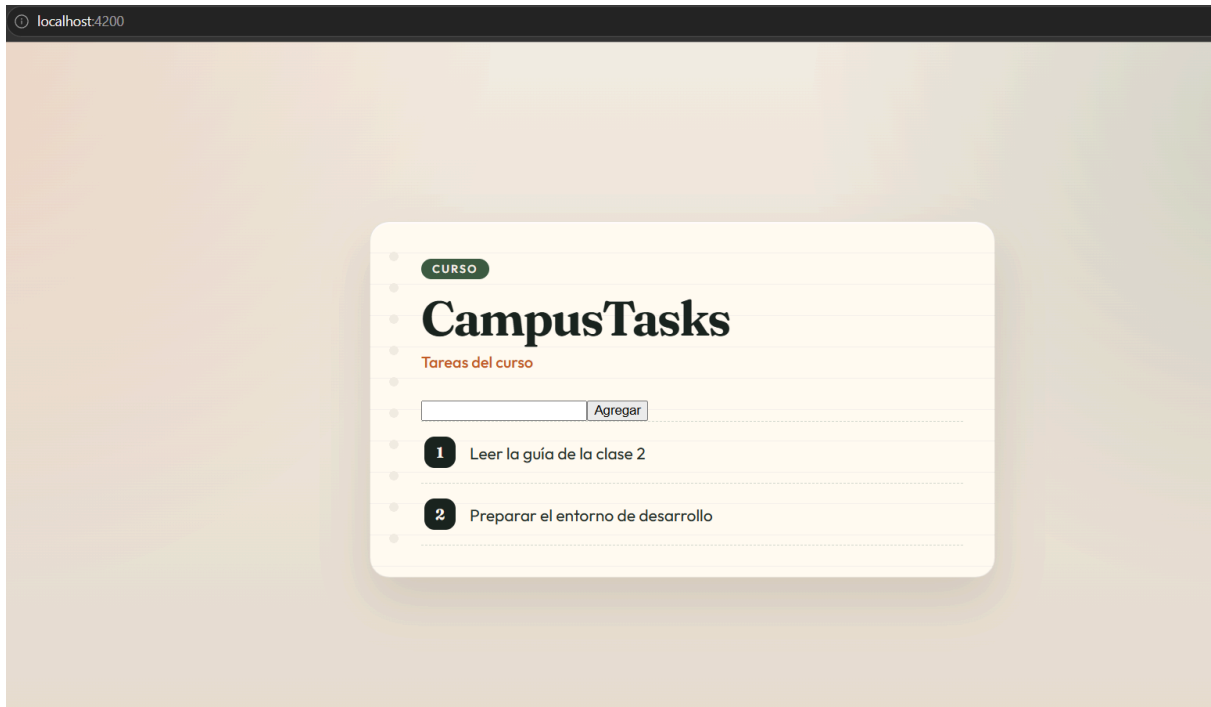
> frontend@0.0.0 start
> ng serve

Initial chunk files | Names          | Raw size
main.js             | main           | 12.91 kB |
styles.css          | styles         | 572 bytes |
polyfills.js        | polyfills      | 95 bytes  |
                    | Initial total  | 13.58 kB

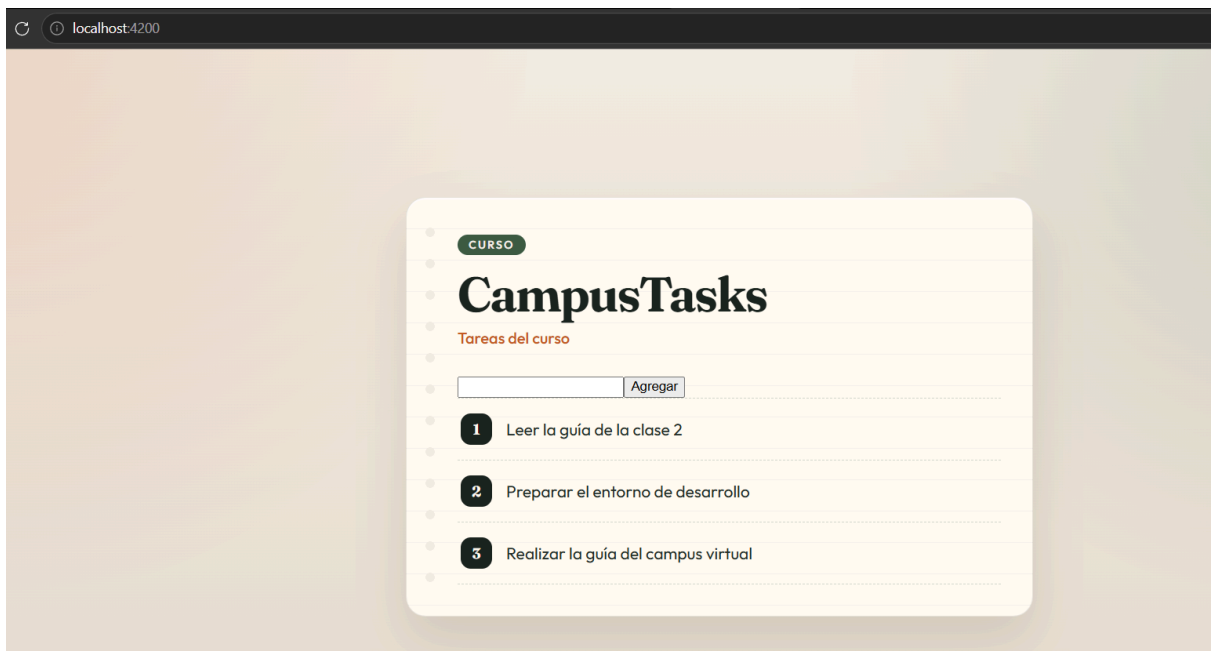
Application bundle generation complete. [2.730 seconds] - 2026-10-02T20:51:35.232Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → press h + enter to show help
```

2. Frontend original



3. Frontend con una nueva tarea adicionada



Paso 8. Ejecutar las pruebas del backend

```
PS C:\Trabajos\Desarrollo 2\campus-task> cd backend
>> npm test

> backend@0.0.1 test
> jest

PASS src/tareas/tareas.service.spec.ts
PASS src/tareas/tareas.integration.spec.ts

Test Suites: 2 passed, 2 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        4.675 s
Ran all test suites.
PS C:\Trabajos\Desarrollo 2\campus-task\backend>
```

Ejecución exitosa de las pruebas del backend mediante Jest (npm test), donde se confirma la aprobación de las 2 suites de prueba (tareas.service.spec.ts y tareas.integration.spec.ts) con 4 casos de prueba superados correctamente.

Paso 9. Ejecutar la prueba del frontend

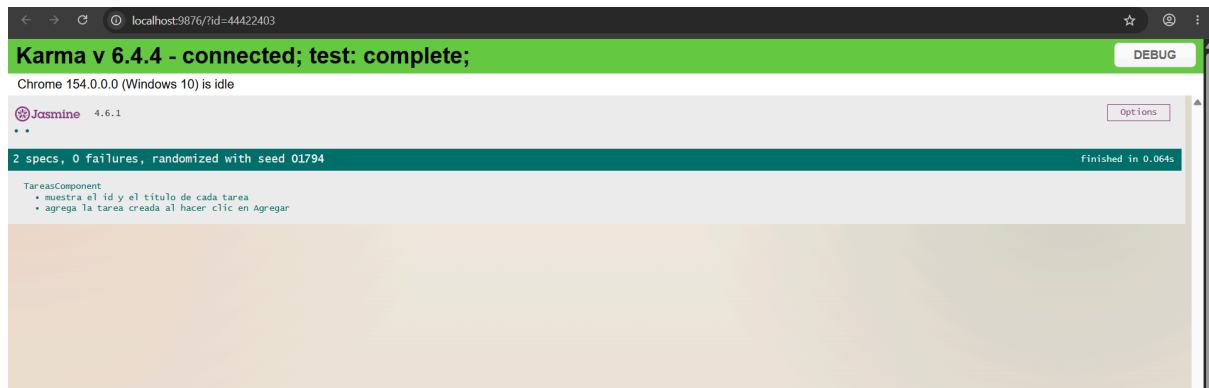
```
PS C:\Trabajos\Desarrollo 2\campus-task> cd frontend
>> npm test

Initial chunk files | Names | Raw size
chunk-AEND4VFP.js | - | 2.27 MB |
polyfills.js | polyfills | 1.04 MB |
test_main.js | test_main | 239.61 kB |
spec-tareas.component.spec.js | spec-tareas.component.spec | 104.20 kB |
jasmine-cleanup-1.js | jasmine-cleanup-1 | 67.20 kB |
chunk-3FMBEB3S.js | - | 2.13 kB |
styles.css | styles | 572 bytes |
jasmine-cleanup-0.js | jasmine-cleanup-0 | 519 bytes |
| Initial total | 3.72 MB

Application bundle generation complete. [1.647 seconds] - 2026-10-02T21:00:29.729Z

Watch mode enabled. Watching for file changes...
02 10 2026 16:00:30.016:WARN [karma]: No captured browser, open http://localhost:9876/
02 10 2026 16:00:30.045:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
02 10 2026 16:00:30.045:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
02 10 2026 16:00:30.060:INFO [launcher]: Starting browser Chrome
02 10 2026 16:00:30.843:INFO [Chrome 154.0.0.0 (Windows 10)]: Connected on socket HpckEgwQWxEMfTs-AAA
A with id 44422403
Chrome 154.0.0.0 (Windows 10): Executed 2 of 2 SUCCESS (0.065 secs / 0.054 secs)
TOTAL: 2 SUCCESS
```

Verificación exitosa del frontend con Karma y Jasmine, donde la aplicación compila correctamente y aprueba la totalidad de las pruebas (2 de 2) ejecutadas en el navegador Chrome.



Interfaz de Karma indicando la ejecución exitosa y sin errores de las 2 pruebas del componente (TareasComponent) en el navegador Chrome.

Antes de comenzar el taller: Lectura de los ejemplos.

Revisamos los tres archivos de prueba del repositorio. En `tareas.service.spec.ts` las pruebas unitarias llaman directamente a listar y crear; `DatabaseService` se reemplaza por un objeto simulado cuyo `query` es un `jest.fn()`, así que no se necesita base de datos. Se verifica el valor devuelto y los argumentos exactos con los que se llamó a `query`. En `tareas.integration.spec.ts` se monta un `TestingModule` con el controlador y el servicio, y con `supertest` se hacen peticiones HTTP reales a `GET /tareas` y `POST /tareas`; se comprueba el código de estado, el cuerpo y los argumentos de `query`. En `tareas.component.spec.ts` se monta `TareasComponent` con Jasmine y Karma, el servicio HTTP se reemplaza por un `spy`, se simulan clics y se verifica que el `spy` se llamó con lo esperado y que la pantalla muestra el resultado.

El patrón que reutilizamos en todas las pruebas nuevas es preparar (definir qué devuelve el `query` o el `spy`), ejecutar (llamar al método, hacer la petición o simular el clic) y verificar (comprobar el resultado y los argumentos).

Desarrollo del taller

1. Backend: Lo que hace el servicio.

[Tareas.service.ts](#) en el backend con los métodos listar y crear existentes y los nuevos actualizar y eliminar.

```
async listar(): Promise<Tarea[]> {
  const resultado = await this.db.query<Tarea>(
    'SELECT id, titulo FROM tareas ORDER BY id',
  );
  return resultado.rows;
}

async crear(titulo: string): Promise<Tarea> {
  const resultado = await this.db.query<Tarea>(
    'INSERT INTO tareas (titulo) VALUES ($1) RETURNING id, titulo',
    [titulo],
  );
  return resultado.rows[0];
}

async actualizar(id: number, titulo: string): Promise<Tarea> {
  const resultado = await this.db.query<Tarea>(
    'UPDATE tareas SET titulo = $1 WHERE id = $2 RETURNING id, titulo',
    [titulo, id],
  );
  return resultado.rows[0];
}

async eliminar(id: number): Promise<Tarea> {
  const resultado = await this.db.query<Tarea>(
    'DELETE FROM tareas WHERE id = $1 RETURNING id, titulo',
    [id],
  );
  return resultado.rows[0];
}
```

2. Backend Servicio

En `tareas.service.spec.ts` agregamos dos pruebas unitarias con el mismo patrón de las existentes. En `actualizar`, preparamos `query` para que devuelva la fila actualizada, llamamos a `service.actualizar(1, 'Título editado')` y verificamos que devuelve esa fila y que `query` recibió el `UPDATE` con el título y el `id` como parámetros. En `eliminar`, preparamos `query` con la fila eliminada, llamamos a `service.eliminar(1)` y verificamos el resultado y que `query` recibió el `DELETE` con el `id`.

```
PS C:\Trabajos\Desarrollo 2\campus-task\backend> npm test

> backend@0.0.1 test
> jest

PASS src/tareas/tareas.service.spec.ts
PASS src/tareas/tareas.integration.spec.ts

Test Suites: 2 passed, 2 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        1.881 s, estimated 3 s
Ran all test suites.
```

Pruebas unitarias de actualizar y eliminar en `tareas.service.spec.ts`, junto con las de listar y crear.

3. Backend Controlador

```
+14ms
[Nest] 25200 - 02/10/2026, 4:55:53 p. m.    LOG [InstanceLoader] DatabaseModule dependencies initiali
lized +0ms
[Nest] 25200 - 02/10/2026, 4:55:53 p. m.    LOG [InstanceLoader] TareasModule dependencies initiali
zed +1ms
[Nest] 25200 - 02/10/2026, 4:55:53 p. m.    LOG [RoutesResolver] TareasController {/tareas/}: +3ms
[Nest] 25200 - 02/10/2026, 4:55:53 p. m.    LOG [RouterExplorer] Mapped {/tareas, GET} route +2ms
[Nest] 25200 - 02/10/2026, 4:55:53 p. m.    LOG [RouterExplorer] Mapped {/tareas, POST} route +0ms
[Nest] 25200 - 02/10/2026, 4:55:53 p. m.    LOG [RouterExplorer] Mapped {/tareas/:id, PATCH} route
+1ms
[Nest] 25200 - 02/10/2026, 4:55:53 p. m.    LOG [RouterExplorer] Mapped {/tareas/:id, DELETE} route
+1ms
[Nest] 25200 - 02/10/2026, 4:55:53 p. m.    LOG [NestApplication] Nest application successfully sta
rted +1ms
█
```

Inicialización exitosa del servidor en NestJS donde se confirman las dependencias de `DatabaseModule` y `TareasModule`, registrando los cuatro endpoints HTTP gestionados por el controlador `TareasController` (`GET /tareas`, `POST /tareas`, `PATCH /tareas/:id` y `DELETE /tareas/:id`) y dejando la aplicación iniciada correctamente para recibir peticiones.

```
PROBLEMS 26 OUTPUT DEBUG CONSOLE TERMINAL PORTS

• PS C:\Trabajos\Desarrollo 2\campus-task> cd backend
• PS C:\Trabajos\Desarrollo 2\campus-task\backend> npm test

> backend@0.0.1 test
> jest

PASS src/tareas/tareas.integration.spec.ts
PASS src/tareas/tareas.service.spec.ts

Test Suites: 2 passed, 2 total
Tests:      8 passed, 8 total
Snapshots:  0 total
Time:       1.326 s
Ran all test suites.
❖ PS C:\Trabajos\Desarrollo 2\campus-task\backend> 
```

Ejecución exitosa de las pruebas del backend mediante Jest (npm test), donde se confirma la aprobación total de las 2 suites de prueba (tareas.integration.spec.ts y tareas.service.spec.ts) con 8 casos de prueba evaluados y superados correctamente.

4. Backend puede probar

a. ciclo crear, editar y eliminar

Con el backend corriendo y conectado a PostgreSQL, probamos las rutas nuevas desde PowerShell con Invoke-RestMethod. Primero consultamos GET /tareas y obtuvimos las dos tareas iniciales. Luego creamos una tarea de prueba con POST /tareas. La editamos con PATCH /tareas/:id enviando { "titulo": "Titulo editado" }, y el backend respondió la tarea con el título nuevo; al consultar GET /tareas el cambio quedó guardado en la base. Después la eliminamos con DELETE /tareas/:id, el backend devolvió la tarea eliminada y una nueva consulta a GET /tareas mostró que la lista volvió a las dos tareas iniciales.

```
PS C:\Trabajos\Desarrollo 2\campus-task\backend> Invoke-RestMethod http://localhost:3000/tareas

id titulo
-- -----
1 Leer la guía de la clase 2
2 Preparar el entorno de desarrollo
3 Realizar la guía del campus virtual

PS C:\Trabajos\Desarrollo 2\campus-task\backend> Invoke-RestMethod -Method Post -Uri http://localhost
:3000/tareas -ContentType "application/json" -Body '{"titulo":"Tarea de prueba"}'

id titulo
-- -----
5 Tarea de prueba

PS C:\Trabajos\Desarrollo 2\campus-task\backend> Invoke-RestMethod -Method Patch -Uri http://localhos
t:3000/tareas/5 -ContentType "application/json" -Body '{"titulo":"Titulo editado"}'

id titulo
-- -----
5 Titulo editado

PS C:\Trabajos\Desarrollo 2\campus-task\backend> Invoke-RestMethod http://localhost:3000/tareas

id titulo
-- -----
1 Leer la guía de la clase 2
2 Preparar el entorno de desarrollo
3 Realizar la guía del campus virtual
5 Titulo editado
```

Prueba manual del backend con la base real: se crea una tarea, se edita con PATCH y se elimina con DELETE.

b. Patch y Delete

Probamos el caso de error con ids que no existen en la tabla. Enviamos PATCH /tareas/999 y DELETE /tareas/999 con curl.exe. En ambos casos la consulta con RETURNING no devuelve filas, el servicio devuelve undefined y el controlador lanza NotFoundException, por lo que el backend responde HTTP/1.1 404 Not Found con el cuerpo {"message":"La tarea 999 no existe","error":"Not Found","statusCode":404}.

```
PS C:\Trabajos\Desarrollo 2\campus-task\backend> Invoke-RestMethod -Method Delete -Uri http://localhost:3000/tareas/5
id titulo
-- -----
5 Titulo editado

PS C:\Trabajos\Desarrollo 2\campus-task\backend> curl.exe -i -X PATCH http://localhost:3000/tareas/999 -H "Content-Type: application/json" -d '{"titulo":"X"}'
HTTP/1.1 404 Not Found
X-Powered-By: Express
Access-Control-Allow-Origin: http://localhost:4200
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 73
ETag: W/"49-tp/crZumG9cEqNW0SIp+orVbeRQ"
Date: Fri, 02 Oct 2026 22:23:29 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"La tarea 999 no existe","error":"Not Found","statusCode":404}
PS C:\Trabajos\Desarrollo 2\campus-task\backend> curl.exe -i -X DELETE http://localhost:3000/tareas/999
HTTP/1.1 404 Not Found
X-Powered-By: Express
Access-Control-Allow-Origin: http://localhost:4200
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 73
ETag: W/"49-tp/crZumG9cEqNW0SIp+orVbeRQ"
Date: Fri, 02 Oct 2026 22:23:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"La tarea 999 no existe","error":"Not Found","statusCode":404}
PS C:\Trabajos\Desarrollo 2\campus-task\backend>
```

PATCH y DELETE sobre un id inexistente responden 404 con el mensaje "La tarea 999 no existe".

5. Creación de rama

```
• PS C:\Trabajos\Desarrollo 2\campus-task> git checkout -b taller/valentina-uribe-aura
Switched to a new branch 'taller/valentina-uribe-aura'
• PS C:\Trabajos\Desarrollo 2\campus-task> git branch
  master
* taller/valentina-uribe-aura
```

```

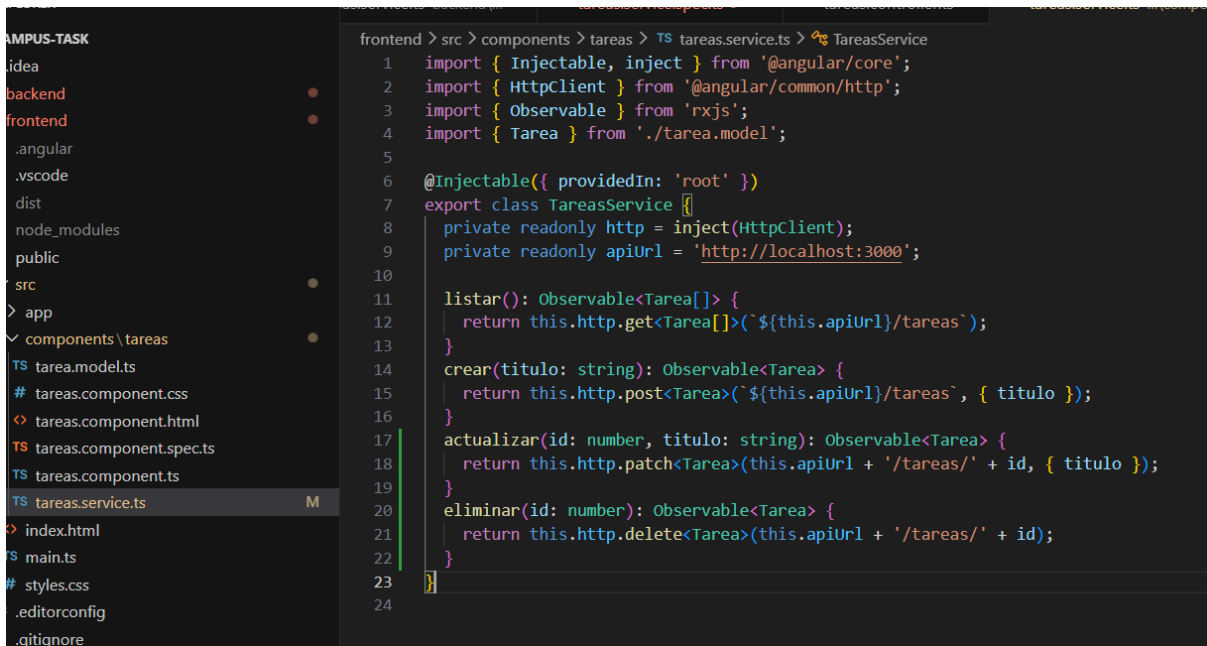
>> git commit -m "Agrega actualizar y eliminar tareas en el backend con pruebas unitarias y de integraci
● acción"Trabajos\Desarrollo 2\campus-task>
[taller/valentina-uribe-aura b4e6612] Agrega actualizar y eliminar tareas en el backend con pruebas u
nitarias y de integraci
4 files changed, 99 insertions(+), 2 deletions(-)
● PS C:\Trabajos\Desarrollo 2\campus-task> git log --oneline -1
b4e6612 (HEAD -> taller/valentina-uribe-aura) Agrega actualizar y eliminar tareas en el backend con p
ruebas unitarias y de integración
● PS C:\Trabajos\Desarrollo 2\campus-task> git status
On branch taller/valentina-uribe-aura
nothing to commit, working tree clean
○ PS C:\Trabajos\Desarrollo 2\campus-task>

```

Registro exitoso del commit en Git dentro de la rama taller/valentina-uribe-aura, guardando los cambios de las funciones de actualizar y eliminar tareas en el backend con sus pruebas, y verificando con git log y git status que el directorio de trabajo quedó limpio.

6. Frontend editar y eliminar desde lista.

Captura de tareas.service.ts con los métodos listar, crear, actualizar y eliminar.



```

frontend > src > components > tareas > TS tareas.service.ts > TareasService
1  import { Injectable, inject } from '@angular/core';
2  import { HttpClient } from '@angular/common/http';
3  import { Observable } from 'rxjs';
4  import { Tarea } from './tarea.model';
5
6  @Injectable({ providedIn: 'root' })
7  export class TareasService {
8      private readonly http = inject(HttpClient);
9      private readonly apiUrl = 'http://localhost:3000';
10
11      listar(): Observable<Tarea[]> {
12          return this.http.get<Tarea[]>(`${this.apiUrl}/tareas`);
13      }
14      crear(titulo: string): Observable<Tarea> {
15          return this.http.post<Tarea>(`${this.apiUrl}/tareas`, { titulo });
16      }
17      actualizar(id: number, titulo: string): Observable<Tarea> {
18          return this.http.patch<Tarea>(this.apiUrl + '/tareas/' + id, { titulo });
19      }
20      eliminar(id: number): Observable<Tarea> {
21          return this.http.delete<Tarea>(this.apiUrl + '/tareas/' + id);
22      }
23  }
24

```

tareas.service.ts del frontend con los métodos nuevos actualizar (PATCH) y eliminar (DELETE).

7. Frontend pruebas de componentes.

Pruebas del frontend completas: Executed 5 of 5 SUCCESS y TOTAL: 5 SUCCESS. Son tus 2 pruebas originales (listar y agregar) más las 3 nuevas (editar, eliminar y error)

```
PS C:\Trabajos\Desarrollo 2\campus-task\frontend> npm test

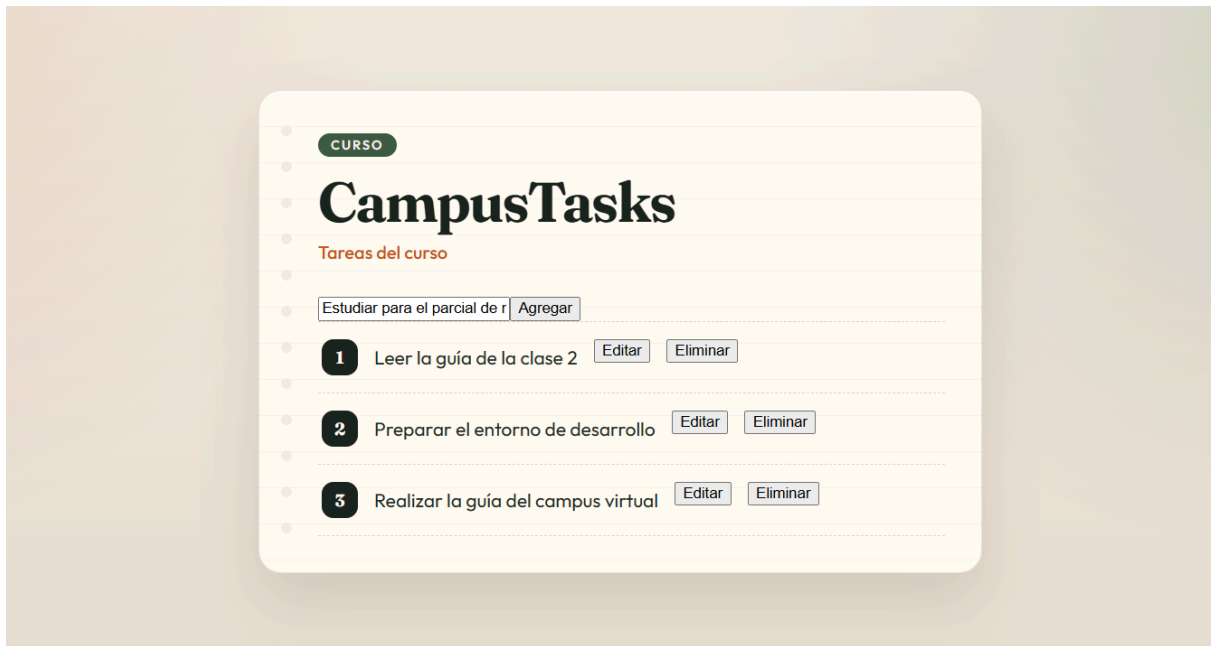
Application bundle generation complete. [1.579 seconds] - 2026-10-02T22:52:41.136Z

Watch mode enabled. Watching for file changes...
02 10 2026 17:52:41.428:WARN [karma]: No captured browser, open http://localhost:9876/
02 10 2026 17:52:41.491:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
02 10 2026 17:52:41.491:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
02 10 2026 17:52:41.508:INFO [launcher]: Starting browser Chrome
02 10 2026 17:52:42.163:INFO [Chrome 154.0.0.0 (Windows 10)]: Connected on socket JryaI53rg1buDrVeAAA
A with id 96313529
Chrome 154.0.0.0 (Windows 10): Executed 5 of 5 SUCCESS (0.096 secs / 0.082 secs)
TOTAL: 5 SUCCESS
```

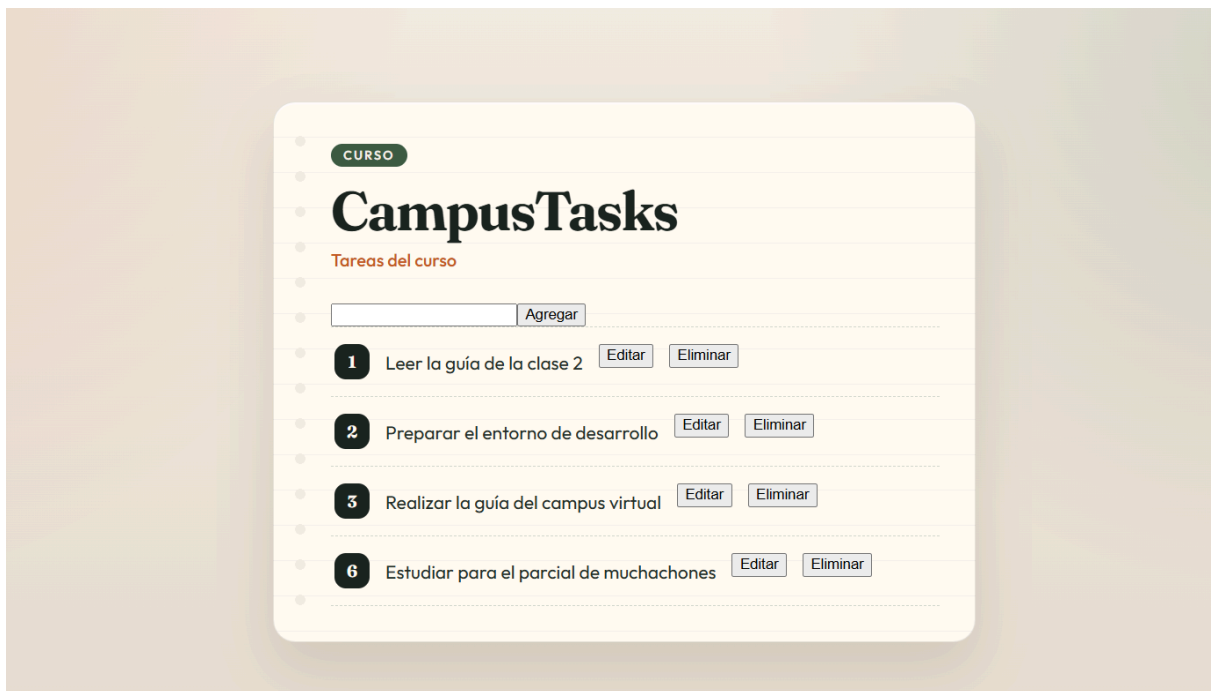
Npm test en frontend: pasan las 5 pruebas de tareas.component.spec.ts (listar, agregar, editar, eliminar y error).



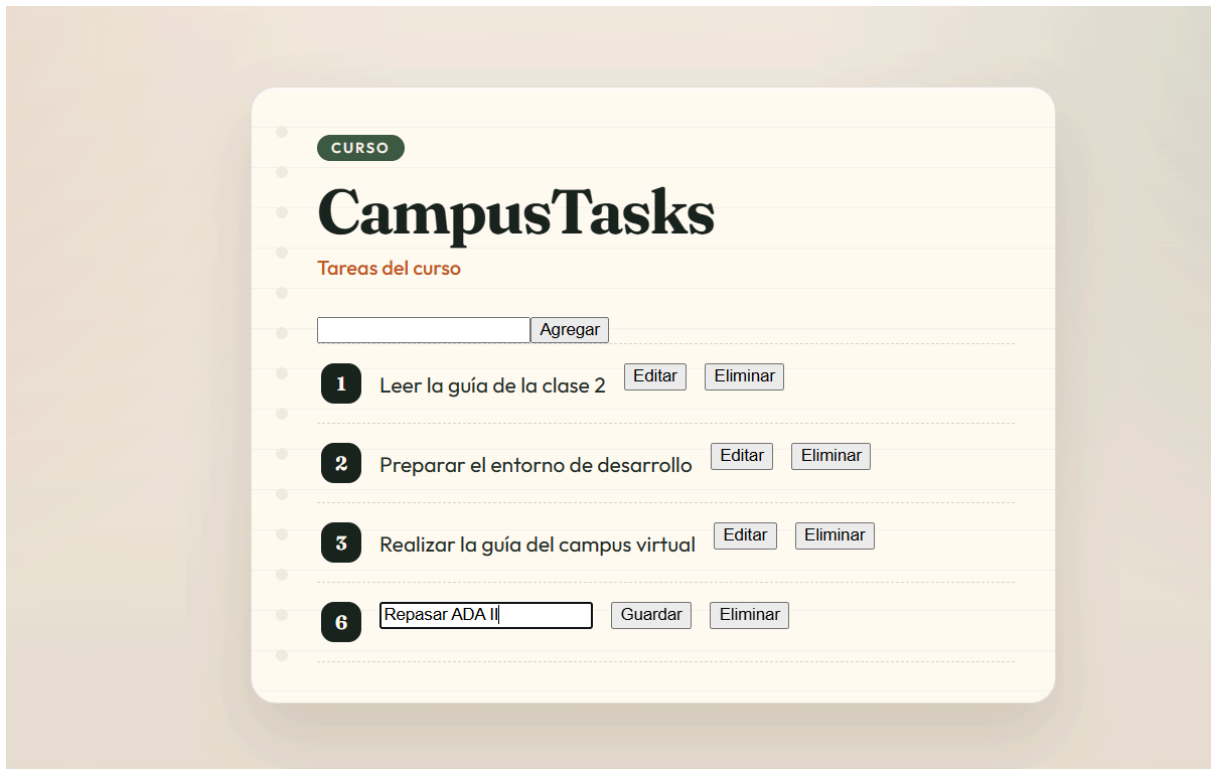
Interfaz web de la aplicación Angular (CampusTasks) desplegada en el navegador, mostrando el formulario de registro y la lista de tareas con sus opciones de edición y eliminación.



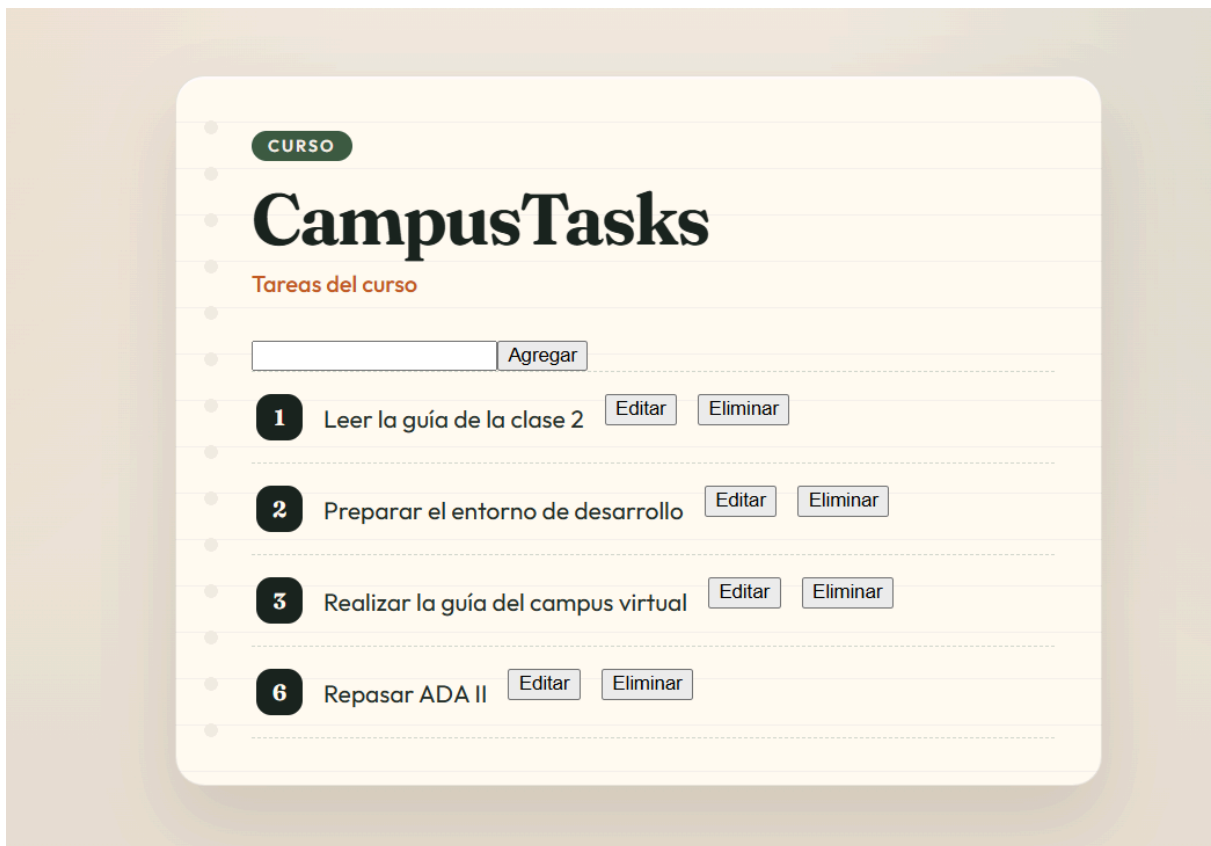
Interfaz web de la aplicación Angular (CampusTasks) mostrando la escritura de una nueva tarea («Estudiar para el parcial de r») en el campo de texto previo a su creación, junto al listado de tareas registradas.



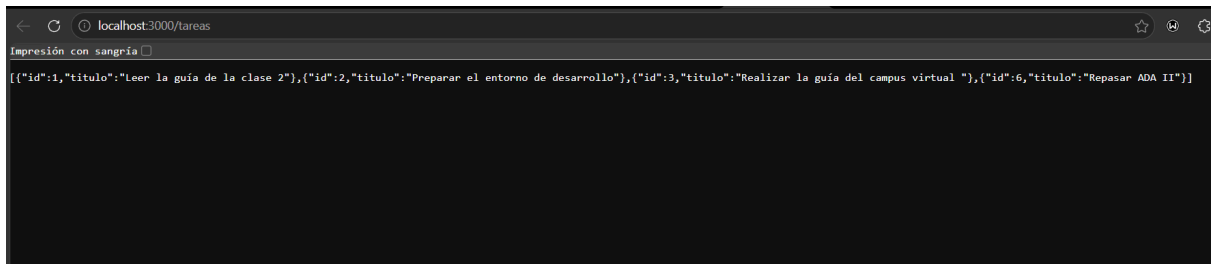
Interfaz web de la aplicación Angular (CampusTasks) tras agregar exitosamente la nueva tarea («Estudiar para el parcial de muchachones»), la cual se muestra integrada en la lista con ID 6 junto al campo de texto reiniciado.



Interfaz web de la aplicación Angular (CampusTasks) en modo de edición para la tarea con ID 6, mostrando el cambio de título a «Repasar ADA II» junto a las opciones de guardar los cambios o cancelar/eliminar.



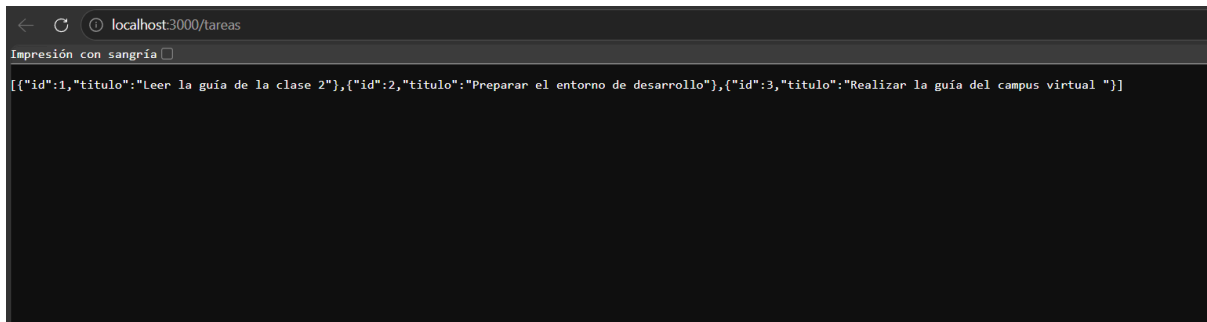
Interfaz web de la aplicación Angular (CampusTasks) tras guardar los cambios, mostrando la actualización exitosa del título de la tarea con ID 6 a «Repasar ADA II» en el listado principal.



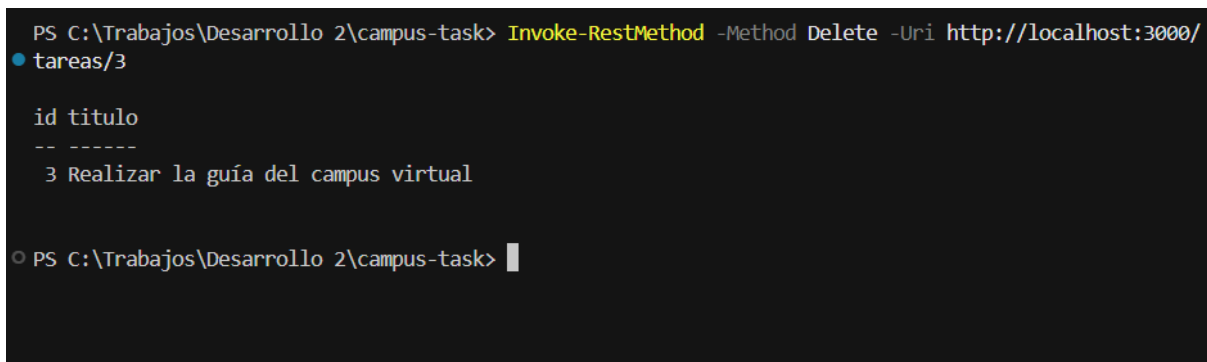
Respuesta en formato JSON obtenida tras consultar el endpoint GET <http://localhost:3000/tareas> del backend en NestJS desde el navegador, confirmando la persistencia y entrega de la lista completa de tareas con los cambios actualizados.



Interfaz web de la aplicación Angular (CampusTasks) tras la eliminación exitosa de la tarea con ID 6, mostrando el listado de tareas actualizado únicamente con los registros principales (ID 1, 2 y 3).



Respuesta en formato JSON obtenida tras consultar el endpoint GET `http://localhost:3000/tareas` del backend en NestJS desde el navegador, confirmando la remoción definitiva de la tarea eliminada y retornando únicamente los registros principales (ID 1, 2 y 3).



Ejecución de una petición DELETE mediante el comando Invoke-RestMethod en PowerShell hacia el endpoint `http://localhost:3000/tareas/3`, confirmando la eliminación de la tarea con ID 3 y retornando la información del registro procesado.



Documentación del proyecto sobre la prueba de manejo de errores en la interfaz de Angular (CampusTasks), incluyendo la captura del mensaje de advertencia y los pasos para reproducir el fallo utilizando Invoke-RestMethod en PowerShell.

```
• PS C:\Trabajos\Desarrollo 2\campus-task> git branch
  master
  * taller/valentina-uribe-aura
• PS C:\Trabajos\Desarrollo 2\campus-task> git status
On branch taller/valentina-uribe-aura
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   frontend/src/components/tareas/tareas.component.html
    modified:   frontend/src/components/tareas/tareas.component.spec.ts
    modified:   frontend/src/components/tareas/tareas.component.ts
    modified:   frontend/src/components/tareas/tareas.service.ts

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Trabajos\Desarrollo 2\campus-task> git add frontend/src
>> git commit -m "Agrega editar y eliminar tareas en el frontend con pruebas de componente"
• >> git log --oneline -3
[taller/valentina-uribe-aura 3629b26] Agrega editar y eliminar tareas en el frontend con pruebas de componente
  4 files changed, 139 insertions(+), 4 deletions(-)
3629b26 (HEAD -> taller/valentina-uribe-aura) Agrega editar y eliminar tareas en el frontend con pruebas de componente
b4e6612 Agrega actualizar y eliminar tareas en el backend con pruebas unitarias y de integración
84cd62d (origin/master, origin/HEAD, master) Merge branch 'tests'
❖ PS C:\Trabajos\Desarrollo 2\campus-task> |
```

Ejecución de comandos de Git en la terminal de PowerShell dentro de la rama taller/valentina-uribe-aura, donde se verifica el estado de los archivos del frontend (git status), se añaden al área de preparación (git add), se crea un nuevo commit con el mensaje «Agrega editar y eliminar tareas en el frontend con pruebas de componente» (git commit) y se confirma el historial reciente del repositorio mediante git log --oneline -3.